

Vision-Based Preharvest Yield Mapping for Apple Orchards

Pravakar Roy · Abhijeet Kislay · Patrick A. Plonski · James Luby ·
Volkan Isler

the date of receipt and acceptance should be inserted later

Abstract We present an end-to-end computer vision system for mapping yield in an apple orchard using images captured from a single camera. Our proposed system is platform independent and does not require any specific lighting conditions. Our main technical contributions are 1) a semi-supervised clustering algorithm that utilizes colors to identify apples and 2) an unsupervised clustering method that utilizes spatial properties to estimate fruit counts from apple clusters having arbitrarily complex geometry. Additionally, we utilize camera motion to merge the counts across multiple views. We verified the performance of our algorithms by conducting multiple field trials on three tree rows consisting of 252 trees at the University of Minnesota Horticultural Research Center. Results indicate that the detection method achieves F_1 -measure .95 – .97 for multiple color varieties and lighting conditions. The counting method achieves an accuracy of 89% – 98%. Additionally, we report merged fruit counts from both sides of the tree rows. Our yield estimation method achieves an overall accuracy of 91.98% – 94.81% across different datasets.

Keywords Yield estimation · Apple detection · Apple counting · Semi-supervised image segmentation · Machine vision · Unsupervised clustering

Efficient control and management of agricultural farms are essential to cope with growing population and nu-

Pravakar Roy · Abhijeet Kislay · Patrick A. Plonski · Volkan Isler

Robotic Sensor Networks (RSN) Lab, Department of Computer Science and Engineering, University of Minnesota
E-mail: {royxx268, kislay004, plons005, isler}@umn.edu

James Luby
Department of Horticultural Science, University of Minnesota
E-mail: lubyx001@umn.edu

merous environmental and economic issues. Precision agriculture techniques provide farmers with methods to monitor the status of their crops on demand and enable them to make crucial decisions (trimming, pruning, application of fertilizers and pesticides etc.). For many commodity crops (rice, corn, wheat etc.), these techniques are already mature enough to provide on-demand maps (yield, crop stress, water quality etc.) as well as help create an infrastructure to implement crucial tasks. However, for specialty crops (such as fruits, vegetables, and flowers), these techniques are still evolving. Specialty crops are particularly good candidates for precision farming and phenotyping studies, because of their species diversity, high value, high management cost, and high variability in growth. One of the main bottlenecks for both these applications is the lack of a convenient yield mapping system.

The variability in plant size, color, shape etc., make it difficult to develop a general yield mapping method. Instead, researchers developed yield mapping systems specific to each crop (Senthilnath et al. (2016); Wang et al. (2013); Das et al. (2015); Gongal et al. (2016); Silwal et al. (2014)). In this paper, we focus on apple orchards and present a complete system for yield monitoring. Our system has a couple of distinguishing features:

1. It is platform independent and relies only on images captured from a single camera. The camera can be mounted on a ground robot or an Unmanned Aerial Vehicle (UAV). It can also be handheld.
2. It does not require any special lighting condition. It operates at daytime in a completely natural environment that is convenient for field applications.

We have two main technical contributions in this paper. 1) We present a novel semi-supervised segmenta-



Fig. 1: Conditions typical to orchard settings. Our algorithms perform well in the first four cases. For the final image, it requires a significant amount of user supervision to obtain reasonable performance.

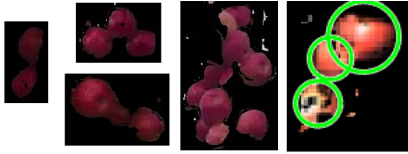


Fig. 2: From left to right: Segmented images exhibiting complex geometry. Sample output of our algorithm on an individual cluster.

tion method to separate the apple pixels from others in the input images. Varying colors of apples (red, golden, green, yellow along with the mixture of their shades), different lighting conditions (based on the position of the sun, clouds etc.), shadows and occlusions created by leaves and branches make the job of identifying regions belonging to apples in an image difficult. We present a semi-supervised segmentation method (Fig. 7) which utilizes minimal user interaction to train a classification model and uses it to identify the apples. Additionally, the system is capable of storing trained models that can be used for a similar variety of apples and lighting conditions. A key aspect of our segmentation method is that it does not rely on detecting every apple in a single frame (Fig. 16(b)). We utilize multiple views for detection and counting based on the assumption that for every apple there is “some” view from which it can be detected. This provides robustness against false positives (caused by shadows and specularities) and helps us achieve high precision and recall (Fig. 18,19). We investigate the sensitivity of our detection method with respect to user input, the impact of occlusions, performance for different color varieties and lighting conditions. We observed that except for extreme situations (Fig. 1 rightmost image) most of the difficulties caused by occlusions and lighting conditions can be eliminated with the help of user supervision. Our algorithm achieves F_1 -score of .95 – .97 for different color varieties and lighting conditions outperforming all existing methods.



Fig. 3: Sample output of our algorithm after the counting step. These two images show the difference in the number of pixels occupied by individual apples. Our algorithm works for both cases regardless of any tuning, whereas techniques such as Circular Hough Transform (CHT) need to be tuned for maximum/minimum fruit size for each individual case.

2) We present a novel method for counting the apples from clusters with arbitrarily complex geometry and merge the apple counts across multiple frames utilizing camera motion. Counting apples from a running sequence of images is difficult as apples can be found in arbitrarily shaped clusters in which almost all apples overlap with each other (Fig. 2). Furthermore, because of specularities as well as occlusions due to leaves and branches, some apples are not detectable at all. We present a method for counting apples based on a classic clustering technique: Gaussian Mixture Models (GMM) (Bilmes et al. (1998)). Our method provides both the counts and location of individual apples in an input image. We model each apple with a Gaussian probability distribution function (pdf) and each apple cluster with a mixture of Gaussians. We present a novel heuristic to find the correct number of components (i.e the number of apples) in the mixture model. Additionally, apples detected in individual frames are tracked across frames to obtain accurate counts (Fig. 4). We merge the information obtained from the per frame operations across multiple frames utilizing camera motion (approximated by pairwise homography). We validate our counting performance both individually and coupled with tracking. We observe that the accuracy of the per-frame counting method is 94.4%. When coupled with tracking, we achieve a varying accuracy of 89% – 98% for seven different videos.

One of the main takeaways from this paper is that the number of visible apples from a single side of a row varies a lot from datasets to datasets (40.85% – 79.83%) (See section 5.4). To obtain a consistent estimate of fruit counts, it is essential to have a coherent geomet-

ric representation of the entire tree row. With the help of our recent work (Dong and Isler (2018); Roy et al. (2018)) we report merged apple counts from both sides of fruit tree rows with a varying accuracy of 91.98% – 94.81% across three different datasets.



Fig. 4: Tracking apples across three consecutive images. Leftmost image is the first frame. In consecutive frames, apples detected earlier are shown in blue. New apples are circled green.

The rest of the paper is organized as follows: in Section 1 we discuss relevant works in literature. In Section 2 we formalize our problem definition and provide a brief overview of our entire computer vision system. Section 3, 4 present our segmentation and counting methods. Section 5 presents the experimental results. Finally, Section 6 presents the conclusion and future work. We start with the related works in the next section.

1 Related Work

There has been a significant recent activity for automating yield estimation (Wang et al. (2013); Das et al. (2015); Hung et al. (2015); Gongal et al. (2016)) in apple orchards. While some of these existing works focus on the entire yield estimation systems, others focus on specific components only. In this section, we first discuss complete yield estimation systems and then focus on individual components. Besides apples, our discussion includes yield estimation systems and components for fruits similar to apples such as citrus, pomegranates, tomatoes etc.

Complete Yield Estimation Systems: Wang et al. (2013) presented a complete system for apple yield estimation. Their system used a side facing, wide-baseline vertical stereo rig mounted on an autonomous ground vehicle. The system operated in controlled artificial illumination settings at night. It uses flashlights to illuminate the apples and classified them using HSV color thresholds. In addition to count information, the stereo

system extracted fruit sizes. Hung et al. (2015) presented a feature based learning approach for identification of red and green apples and extraction of fruit count. They used a conditional random field classifier to segment fruits using color and depth data. Das et al. (2015) developed a sensor suite for extracting plant morphology, canopy volume, leaf area index and fruit counts that consists of a laser range scanner, multi-spectral cameras, a thermal imaging camera and navigational sensors. They used a Support Vector Machine (SVM) trained on pixel color to locate apples in images. Gongal et al. (2016) developed a new sensor system with an over-the-row platform integrated with a tunnel structure which acquired images from opposite sides of apple trees. The tunnel structure is used to minimize illumination of apples with direct sunlight and to reduce the variability in lighting. In contrast to these systems, our goal is to develop a general computer vision system to extract the count of apples in an orchard row from a monocular camera. The system is independent of any particular hardware platform and can be used with a various type of robotic and non-robotic systems. In the rest of this section, we will discuss systems developed for tackling specific portions of the yield estimation system.

Apple Detection: The problem of locating fruits from captured photographs have been studied extensively in the literature. Early systems relied on hard color thresholds. The early methods relying on machine learning were targeted to learn these thresholds. Jimenez et al. (2000) presented a survey of these early computer vision methods for locating fruits on trees. More recent approaches include Zhou et al. (2012), who presented a method that uses RGB and HSV color thresholds to detect Gala apples. Linker et al. (2012), used K-nearest neighbor (KNN) classifier along with blob and arc fitting to find red and green apples. The authors avoid specular lighting conditions by capturing images close to sunset. Changyi et al. (2015) developed an apple detection method that uses backpropagation neural network (Hecht-Nielsen (1989)) trained on color features. Similar to them, Liu et al. (2016) presented a method for detecting apples at night using pixel color and position as features in neural networks. The earliest work close to our method is (Tabb et al. (2006)) who developed a method for detecting apples from videos using background modeling by global Gaussian mixture models (GMM). The method relied on images collected using an over-the-row harvester platform that provides a consistent background and illumination setting. In contrast, we operate in natural settings, use GMM for unsupervised clustering in every image and classify them using pre-trained models.

Deep Learning for Detecting Apples: Recent advancements in deep learning inspired researchers to apply these techniques for identifying apples. [Bargoti and Underwood \(2017\)](#) used Multi-Scale Multi-Layered Perceptron network (MLP) for apple and almond detection. [Stein et al. \(2016\)](#) used a similar deep learning technique to identify mangoes. [Chen et al. \(2017\)](#) used a Fully Convolutional Neural Network ([Long et al. \(2015\)](#)) for apple segmentation. Though deep learning methods are accurate, they require a large amount of training data [e.g we trained a Fully Convolutional Network (FCN) using the data from ([Bargoti and Underwood \(2017\)](#)) and the network did not perform well on our data. When we use some of our data for training though, performance improved drastically]. Generating such training data for different varieties of apples, in different lighting conditions are tedious and cumbersome. In contrast, our method generalizes to any variety of apples and different lighting conditions (as long as apples can be distinguished from other objects by color) using a modest amount of user assistance.

Registering Fruits Across Images: Compared to locating fruits in images, the studies on registering fruits across multiple images has been limited. Only the systems with full yield estimation pipelines studied this problem. [Wang et al. \(2013\)](#) used stereo cameras and point cloud alignment (using odometry and GPS) to avoid double counting. They aligned the apples in 3D space and removed the ones which are within five centimeters of a previously registered apple. [Hung et al. \(2015\)](#) used sampling at certain intervals to remove overlap between images. [Das et al. \(2015\)](#) used optical flow and navigational sensors to avoid duplicate apples. In our previous work ([Roy and Isler \(2016\)](#)), we presented a novel method for registering apples based on affine tracking ([Baker and Matthews \(2004\)](#)) and incremental structure from motion (SfM) ([Sinha et al. \(2014\)](#)). In this work, we are using homography between frames to track fruits and to avoid double counting.

To obtain an accurate yield estimate, in addition to registering fruits from a single side, we need to register the fruits from both sides of the row. Only [Wang et al. \(2013\)](#) handles this problem and used point cloud alignment using odometry and GPS. In this paper, we utilize a novel technique that utilizes global and local semantic information (ground plane, tree trunks, the silhouette of foliage etc.) for merging the apple counts from both sides ([Dong and Isler \(2018\)](#); [Roy et al. \(2018\)](#)).

Counting Fruits: Most of the yield estimation work described before report counting results. The methods for counting fruits from segmented images are dominated by circular Hough transforms (CHT) ([Silwal et al. \(2014\)](#); [Changyi et al. \(2015\)](#); [Liu et al. \(2016\)](#)). The

main bottleneck of using CHT is that the parameters need to be tuned across different datasets. Another issue with this method is occlusion. Due to occlusion many apples are not fully visible and cannot be approximated by a circle. Different from these methods, [Senthilnath et al. \(2016\)](#) used K-means, a mixture of Gaussians and Self-Organizing Maps (SOM) to detect individual tomatoes within a cluster. They used the Bayesian Information Criterion (BIC) ([Chen and Gopalakrishnan \(1998\)](#)) to select the optimal number of components in these methods. [Chen et al. \(2017\)](#) used a neural network for determining the count of the apples. In contrast, we use a Gaussian mixture model to count the apples from registered and segmented apple clusters. Unlike CHT, our method does not require any parameter tuning and can handle arbitrarily complex clusters. Unlike the neural network based methods, it does not require any training. The method yields competitive results compared to most of the state of the art methods. In this paper, we present a complete computer vision system for yield estimation in apple orchards. We present a novel segmentation method for detecting apples, a tracking method based on homography and a non-parametric counting algorithm. We start with an overview of the entire system in the next section.

2 Problem Formulation and Overview of Our Computer Vision System

In this section, we formalize our problem definition and present a brief overview of our system. We start with the problem definition.

Problem Formulation: Given a set of images from a calibrated monocular camera facing one side of a row in an apple orchard (where it is assumed that the variance in viewing direction is very small), we want to compute the total apple counts from the entire image sequence. To solve this problem, we proceed in a per frame manner - separate the apple pixels from others in every frame and count the number of apples in each of them. Afterward, we merge the information across multiple frames by utilizing the approximate camera motion from pairwise homographies. We discuss the components of our system briefly in the rest of this section.

2.1 Segmentation

The segmentation component takes as input a color image for each frame, and outputs a binary mask which marks whether each pixel in the image belongs to the class `apple` (Fig. 7). This component is presented in detail at Section 3.

First, the image is over-segmented into SLIC superpixels (Achanta et al. (2012)), using the LAB color space. A single representative color (mean LAB color of the pixels within the superpixel) is assigned to each superpixel. Then superpixels are clustered by color into approximately 25 color classes. Finally, it is determined for each class whether it describes apples, based on KL divergence (Goldberger et al. (2003)) from hand-labeled classes. These hand-labeled classes can be obtained from the unsupervised clusters of the first few frames of a particular video, to easily account for current lighting conditions and the color of the particular apple variety at its particular ripeness.

2.2 Per Frame Counting

The per frame counting component takes as input the binary segmented mask for each frame, and outputs a set of bounding boxes and associated integers for each frame where each bounding box represents a connected cluster of apples, and the integer is the estimated number of apples in that cluster (Fig. 2, 3). This component is presented in detail at Section 4.1.

First, a connected component analysis is performed on the binary apple mask. Each connected component is examined separately, to determine how many apples it contains. We perform a Gaussian Mixture Model (GMM) based clustering to estimate the number of apples contained within the bounding box, as well as their positions.

2.3 Camera Motion Approximation

The camera motion approximation component takes as input the detected SIFT (Lowe (1999)) features in the original input images. It computes a pairwise homography using the SIFT feature matches.

2.4 Merging the Counts From Multiple Views

The merging component takes as input a sequence of per frame bounding boxes with apple counts, as well as estimated frame-to-frame homographies of the approximately planar scene. The output is a total count of unique apples seen in the frame sequence (Fig. 4, 12). This component is presented in Section 4.2.

Essentially, it propagates the computed bounding boxes forward and recomputes the counts when a bounding box overlaps with another one.

3 Apple Segmentation

In the segmentation stage, the goal is to identify pixels which are likely to belong to an apple. Specifically, the segmentation algorithm takes as input an RGB image and produces a binary mask where pixels belonging to apples are marked as ones and all other pixels are marked as zeros.

One approach to segmentation is to simply choose apple pixels based on predetermined thresholds on color values. While this approach has the advantage of simplicity, in field conditions it often fails due to the variability of lighting conditions. In recent years, deep-learning-based approaches such as convolutional neural networks emerged as powerful, general methods for image classification. Recently their extension to Fully Convolutional Networks (FCN) trained for pixel-wise prediction has gained popularity for fruit and crop segmentation (Chen et al. (2017)). While these models achieve high accuracy, training a network general enough to accommodate variations in light and visibility conditions remains challenging. Further, training FCNs for each orchard video separately requires a significant amount of human labor involved in generating ground truth label for each apple.

We propose a semi-supervised image segmentation method based on color and shape features of apples. By itself, the segmentation algorithm runs at 5 to 6 frames per second on images of size 1920×1080 and 15 frames per second for images of the size of 640×480 on a DELL XPS laptop with 16GB RAM and 2GB GPU memory. The training required from the user is minimal. It is assisted by a simple, convenient user interface. As we show in Section 5.3, the method generalizes well for the cases where the training and testing data are from different portions of an orchard taken on different days. Our method is expected to work for cases where apples are visibly distinguishable from the surrounding vegetation based on the color. Some working and challenging conditions are shown in Fig. 1.

Details of the segmentation process: For each frame in the image sequence, we convert it from RGB to LAB (Gauch and Hsia (1992)) color space and perform SLIC superpixel segmentation (Fig. 7(b)) (Achanta et al. (2012)). This generates a set of superpixels $\mathcal{S}$ for each image.

$$\mathcal{S} = \{\mathbf{s}_1, \dots, \mathbf{s}_N\}$$

Here each superpixel $\mathbf{s}_i$ is represented by $\{\mu_i^L, \mu_i^A, \mu_i^B\}$, the mean L,A,B values for all pixels $\mathbf{s}_i$. We assume that the set $\mathcal{S}$ of superpixels can be modeled as a density function $p(\mathbf{s}|\theta)$ governed by the set of parameters θ .

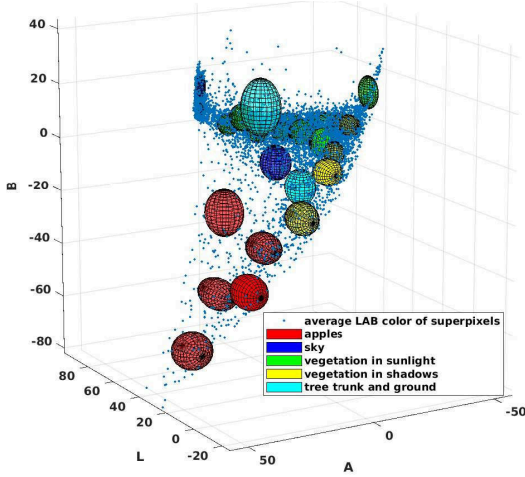


Fig. 5: Gaussian representation of semantic entities. Each blue data point is the mean LAB color for each superpixel in the image. Each 3D ellipsoid represents the Gaussian fit on clusters obtained from unsupervised clustering using EM and GMM. For visualization, showing 19% of the standard deviation for each Gaussian ellipsoid. Red colored ellipsoids are capturing the superpixels belonging to apples.

For soft segmentation, we model $\mathcal{S}$ as a Gaussian Mixture Model (GMM) (Chuang et al. (2001); Ruzon and Tomasi (2000)) with M components. Hence p is represented by a set of Gaussian components G_i and parameters θ as the respective mean μ_i and covariances Σ_i for each G_i .

$$p(\mathbf{s}|\theta) = \sum_{i=1}^M \alpha_i G_i(\mathbf{s}|\mu_i, \Sigma_i)$$

such that $\sum_{i=1}^M \alpha_i = 1$

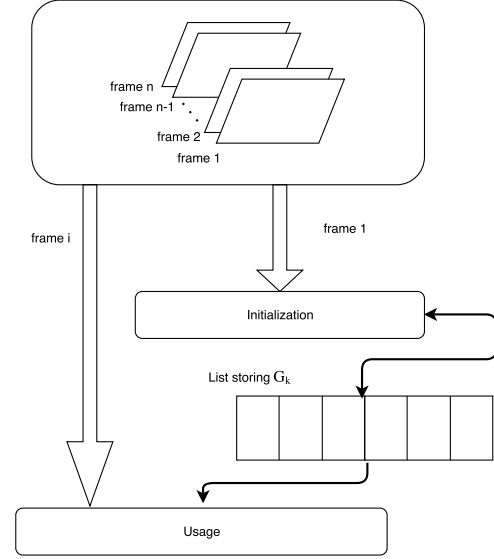
The likelihood function of the parameters can be written as:

$$\mathcal{L}(\mu, \Sigma|\mathcal{S}) = \prod_{i=1}^N p(\mathbf{s}_i|\mu_i, \Sigma_i)$$

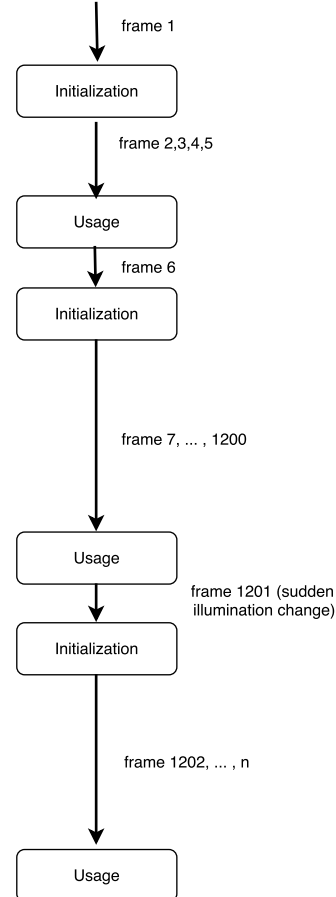
The resultant Gaussian mixture density estimation problem is:

$$(\mu^*, \Sigma^*) = \underset{\mu, \Sigma}{\operatorname{argmax}} \mathcal{L}(\mu, \Sigma|\mathcal{S})$$

where $\mu^* = \mu_1^*, \dots, \mu_M^*$ and $\Sigma^* = \Sigma_1^*, \dots, \Sigma_M^*$. Expectation Maximization (EM) is used (Bilmes et al. (1998)) to estimate μ_i^*, Σ_i^* for each Gaussian cluster G_i . Each Gaussian cluster G_i thus generated contains similar colored superpixels (Fig. 5) and represents different semantic entities of an orchard frame such as the sky, soil, apples, leaves, branches, etc.



(a) Input is a video. For each frame in the initialization step, among all Gaussian components G_i , G_k are chosen and saved in a list using a UI. These saved G_k are used for finding Gaussian components in the usage step.



(b) A general way of using the Usage and Initialization phase for a dataset.

Fig. 6: Explanation of the user interface.

Next, we identify which among these G_i capture superpixels belonging to apples. We divide this step into two parts: (1) *initialization* and (2) *usage* (Fig. 6(a)). In the *initialization* phase, we provide an interface for a user to interact with the G_i . Here the user is allowed to:

- select components G_k which completely capture apples. For each selected G_k , their respective μ_k^* and Σ_k^* are pushed in a list in memory.
- delete components from the list stored in memory. This step is generally needed where there is a sudden illumination change and the previous stored components G_k become bad (Fig. 6(b)). We can update the old list, and continue the segmentation process from the current frame.

In the *usage* phase, the user interface is not invoked. To find components belonging to apples, we perform a simple matching between the current frame’s G_i and the saved G_k . We use KL Divergence as the distance measure for comparing Gaussians from G_i and G_k . For a matched Gaussian G_i , all the superpixels within the 90% confidence bounds are identified as superpixels belonging to apples. A normal usage paradigm is shown in Fig. 6(b).

4 Counting Apples and Merging the Counts across Multiple Frames

After the segmentation phase, we count the number of apples from each segmented frame and merge the counts across multiple frames utilizing the estimated camera motion. In this section, we present both of these methods in detail. We start with per frame counting in the following section.

4.1 Per Frame Counting

Given a segmented input image, we would like to find the number and location of all the apples in the image. We use a Gaussian Mixture Models (GMM) based clustering method. Instead of color, we now focus on the spatial components of the image. This method holds numerous advantages over the Circular Hough Transform (CHT) based techniques (Pedersen (2007)) - it does not require manual parameter tuning, can handle a significant level of occlusion and find apples of rapidly varying size. A preliminary version of this algorithm was presented in (Roy and Isler (2017)). In Roy and Isler (2017), we showed the comparison of this method with a baseline method similar to CHT and it outperformed it significantly (the accuracy of baseline method 69%,

the accuracy of GMM based method 91%). In this, paper we further validate the method with ground truth from multiple datasets and implement it in a close to a real-time system performing at 2-3 fps.

In our method, each apple is modeled by a Gaussian probability distribution function (pdf) and apple clusters are modeled as a mixture of Gaussians. We start by converting the input cluster image I to binary. Let this binary image be denoted by I_b . The locations of the non-zero pixels in the binary image are used as input to GMM.

Let X represent the set of apples we are trying to find. Then, we can convert our problem to a Gaussian mixture model formulation in the following way:

$$P(I_b|X) = G^k(\phi, \mu, \Sigma) = \sum_{i=1}^k \phi_i G_i(\mu_i, \Sigma_i) \quad (1)$$

Here, $G^k(\phi, \mu, \Sigma)$ is a Gaussian mixture model with k components, and G_i is the i th component of the mixture. μ_i and Σ_i are the mean and covariance of the i th component. The covariance matrix $\Sigma_i = [\sigma_{x_i}^2, \sigma_{y_i}^2]$ is diagonal. ϕ_i is the weight of the i th component where $\sum_{i=1}^k \phi_i = 1$ and $0 \leq \phi_i \leq 1$.

Given model parameters $\theta = \{\phi, \mu, \Sigma\}$, the problem of finding the location of the center of the apples and their pixel diameters can be formulated as computing the world model which maximizes $P(I_b|X)$.

Each component $G_i(\mu_i, \Sigma_i)$ of the mixture model represents an apple with center at μ_i , equatorial radius $2\sigma_{x_i}$ and axial radius $2\sigma_{y_i}$.

A common technique to solve for $\arg \max P(I_b|X)$ is the expectation maximization (EM) algorithm (Bilmes et al. (1998)). As is well-known, EM provides us a local greedy solution to the problem. Since EM is susceptible to local maxima, initialization is very important. We used K-means++ (Hartigan and Wong (1979)) (which uses randomly selected seeds to avoid local maxima) for initialization of EM.

Selecting the Number of Components: In our problem formulation, the number of components k is the total number of apples in image I . EM enables us to find the optimal location of the apples given the total number of apples k . Our main technical contribution is a method to calculate the correct k . Let the correct number of apples in the input image be κ . We tried different state-of-the-art techniques (Akaike Information Criterion (AIC) (Grunwald (2004)), Minimum Description Length (MDL) (Grunwald (2004)) etc.) for finding κ . None of them worked out of the box for our purposes (Fig. 8). Therefore, we propose a new heuristic for evaluating mixture models with a different number of components based on MDL.

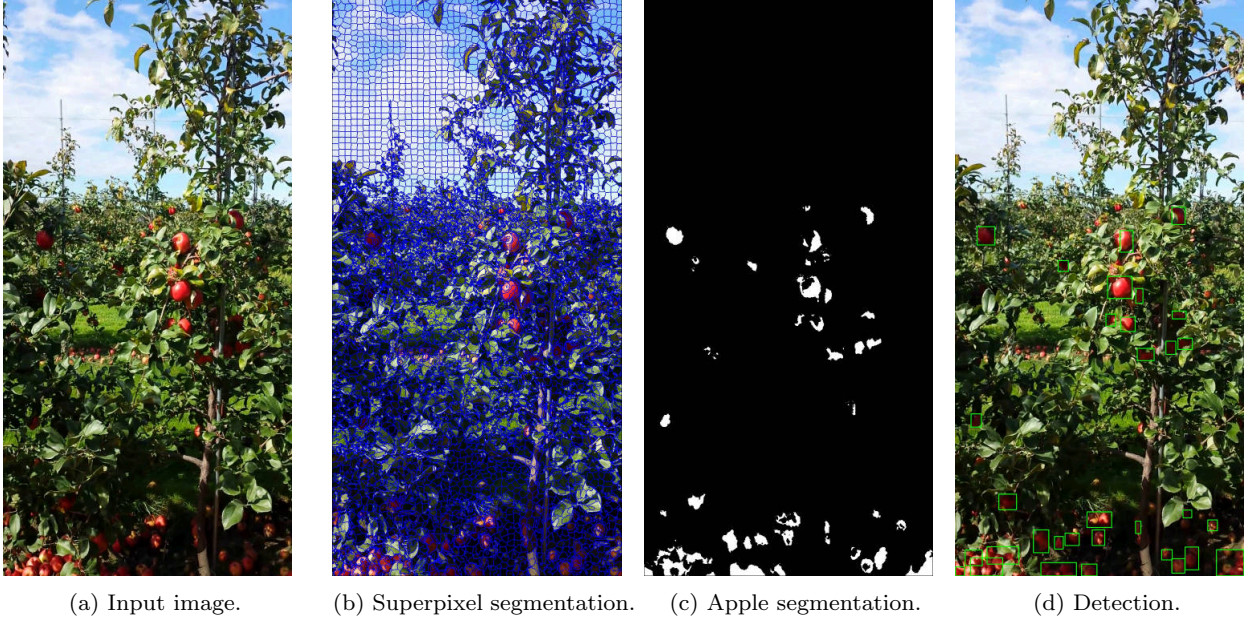


Fig. 7: Segmentation pipeline.

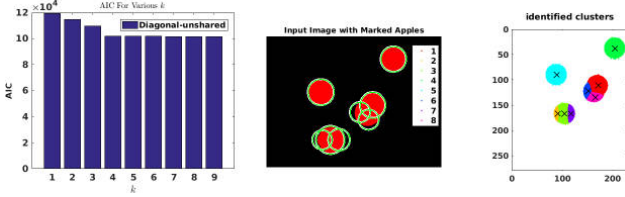


Fig. 8: Popular methods like AIC/BIC do not work out of the box for our purposes. These criteria have a tendency of choosing higher values of k . In this synthetic image, we have only five circles but the AIC based criterion value (Bar plot in the middle) was lowest for $k = 6$ and consequently, it finds eight apples.

Let $\sigma_{min} = \min(\sigma_{x_i}, \sigma_{y_i})$ and $\sigma_{max} = \max(\sigma_{x_i}, \sigma_{y_i})$. Using the mean and covariances of the i th component we define a 2D Gaussian kernel $\mathcal{G}(\mu_i, \sigma_{max})$ where σ_{max} is the variance. Let $P(\mu_i)$ denote the response of the kernel when placed at the center μ_i in the original input image I and C_i denote the total number of pixels clustered by $G_i(\mu_i, \Sigma_i)$. For each component $G_i(\mu_i, \Sigma_i)$, of the mixture model $G^k(\phi, \mu, \Sigma)$ we define the reward R_i in the following way,

$$R_i(G_i) = \phi_i \left[P(\mu_i) + P(\mu_i) \left(\frac{\sigma_{min}}{\sigma_{max}} \right)^2 + P(\mu_i) \frac{C_i}{\pi \sigma_{max} \sigma_{min}} - \frac{1}{3} (\pi \sigma_{x_i} \sigma_{y_i} - C_i) \right] \quad (2)$$

For most of the images, we only capture the frontal views of the apples, which can be easily approximated

by circles lying on a plane. All four terms in equation (2) reward specific spatial characteristics of the Gaussian pdf related to this fact. $P(\mu_i)$ represents the strength of the distribution in terms of pixel values and is present in the first three terms. The second term rewards circular shaped distributions using the eccentricity of the pdf. As the eccentricity $\epsilon = \sqrt{1 - \frac{\sigma_{min}^2}{\sigma_{max}^2}}$ for circles is zero, we use $1 - \epsilon^2 = \left(\frac{\sigma_{min}}{\sigma_{max}} \right)^2$ as the rewarding factor. The third term rewards coverage. The fourth term penalizes Gaussian pdfs covering large area and clustering very few points.

Now if we find out the reward $R_i(G_i(\mu_i, \Sigma_i))$ for all the components k , the total reward for the mixture model $G^k(\phi, \mu, \Sigma)$ can be computed by summing them together.

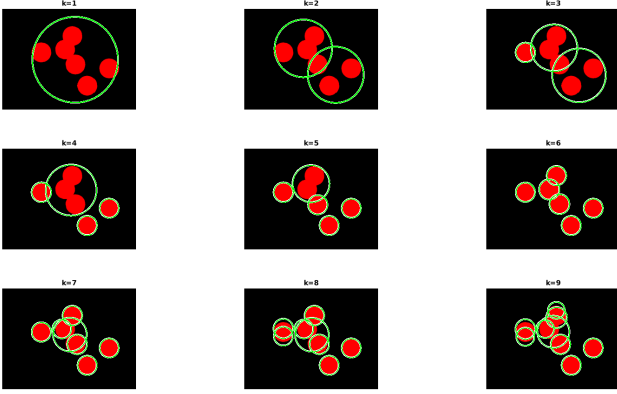
Next, we define the penalty term. The traditional MDL penalty term is $U = cp \log(|Y|)$ where p is the number of parameters in the model, $|Y|$ is the total size of the input data, and $c = \frac{1}{2}$ is a constant. Based on this principle, our penalty term is $V(G^k(\phi, \mu, \Sigma))$ is defined as the following

$$V(G^k(\phi, \mu, \Sigma)) = c'(3k) \log \left(\sum_{\mathbf{x}} (I_b(\mathbf{x}) \neq 0) \right) \quad (3)$$

where x represents the pixel index across the image I_b . Compared to the traditional MDL based penalty we have the constant $c' = \frac{3}{2}$ instead of $c = \frac{1}{2}$. This is attributed to the fact that the reward expression (2) has three terms compared to one. The number of components k is multiplied by three as each Gaussian has

three parameters $[\mu_i, \sigma_{x_i}, \sigma_{y_i}]$. With these terms defined, we choose the correct number of components κ in the following way:

$$\kappa = \arg \max_k R(G^k(\phi, \mu, \Sigma)) - V(G^k(\phi, \mu, \Sigma)) \quad (4)$$



(a) Predicted circles for different number of components in GMM.

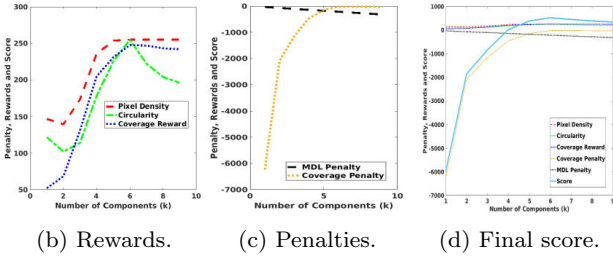


Fig. 9: A synthetic example consisting of six random circles and plots illustrating how the number of components are selected. Figure (a) shows how the pdfs cover the circles for different values of k . Figure (b)-(d) shows the score calculated from the rewards and penalties following the right hand side of equation (4). The plot shows that the score is maximum for $k = 6$ which is indeed the correct number of components.

To have a better understanding of the selection procedure, we demonstrate a synthetic example at Fig. 9(a). From Fig. 9(b), it is evident that except for $k = 6$, other mixtures have low circularity. The coverage rewards and pixel density components increase with k and converge to a steady state. While the penalty for minimum description length principle increases with k , generally the penalty for coverage decreases with k . In this example, the crucial factors in determining the score are circularity and the coverage penalty. For $k = 6$ circularity is at the peak and coverage penalty is lowest and consequently, the score was maximum for $k = 6$. The plot of the corresponding rewards, penalties and final scores

are shown in Fig. 9(b) - (d). We show sample results from our datasets in Fig. 10.

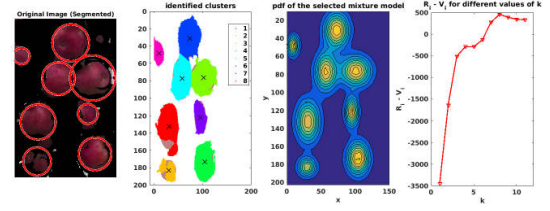


Fig. 10: A sample output from GMM on real data.

After executing the counting algorithm on each of the bounding boxes we have the location of the apples within each box and the number of apples per box. This is the input to our merging method that merges the apple counts across multiple frames. We describe this procedure in details in the next section.

4.2 Merging Count Across Multiple Frames

The per-frame counting method provides us with the apple counts for a single frame. In natural settings though, the visibility of a particular cluster can change drastically with a change in camera position. A cluster might be completely invisible/partially visible from a particular view and yet clearly visible from other views. For these reasons, our segmentation method may not be able to obtain the perfect segmentation in each frame and consequently, the predicted number of apples might be incorrect. Therefore, to obtain the correct apple count for each cluster we merge the counts from different frames. For this operation, we need to establish the correspondence between the clusters across multiple frames. It is executed by utilizing the camera motion.

As our camera viewing direction does not change much, and the scene is roughly planar, we model the camera motion between consecutive frames by pairwise homography (Eshel and Moses (2008)). The homography between frames is estimated by matching SIFT (Lowe (1999)) features above the ground. Using homography we will keep track of the boxes generated by the connected component analysis across the entire image sequence (Fig. 11).

Let $b_1^1, b_2^1, \dots, b_m^1$ be the bounding boxes generated by connected component analysis for frame 1. Let the apple count for each of these boxes be $c_1^1, c_2^1, \dots, c_m^1$ (computed by the counting method). When we find a bounding box for the first time we initialize a counting list that contains computed counts from the first frame.

Now for frame 2, let the bounding boxes be $b_1^2, b_2^2, \dots, b_n^2$ and the counts be $c_1^2, c_2^2, \dots, c_n^2$. Let the homography

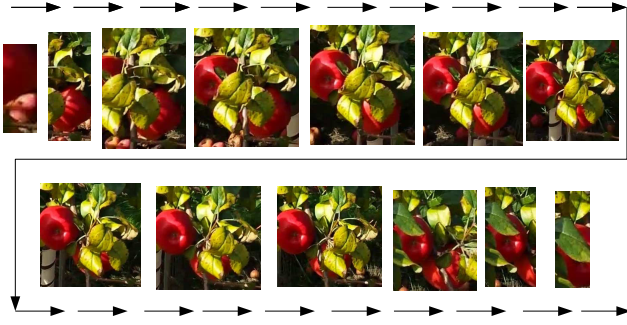


Fig. 11: Tracking an apple cluster over 31 frames. For clarity only 13 frames with significant changes are shown. The arrows indicate the direction of the views from start to end.

that maps frame 1 to frame 2 be 2H_1 . We propagate all the bounding boxes in frame 1 to frame 2 using 2H_1 . This is executed by multiplying the center of each bounding box with 2H_1 . Next, we check the overlap between these propagated bounding boxes and the original bounding boxes on frame 2. If the overlap is more than 10% we assume that these bounding boxes correspond to the same cluster. We will add the apple count to the list initialized previously using the following rules:

- When a bounding box in the current frame does not overlap with any of the propagated bounding boxes, a new counting list for this box is initialized with the current count.
- When only one bounding box in the current frame overlaps with a propagated bounding box, the count from the bounding box in the current frame is added to the counting list for the propagated box
- Otherwise, when two or more bounding boxes overlap with a bounding box from the previous frame, their counts are added to obtain the total count to be recorded in the list. To have a better understanding of this rule, we consider the following scenario: Let b_1^2, b_3^2 be overlapping with b_1^1 . Prior to frame 2 the counting list of b_1^1 , $c_{b_1^1}$ had only one entry, c_1^1 . As there is overlap, a new entry will be inserted. This new entry is $c_1^2 = c_1^1 + c_3^2$.
- The overlapping bounding boxes are unioned to obtain a new bounding box covering all of them. These new bounding boxes will be propagated to the next frame.

At the end of the image sequence, we have a set of unique boxes with count lists. We compute the median for each of the boxes and the sum of these are reported as the total count (Fig. 12).

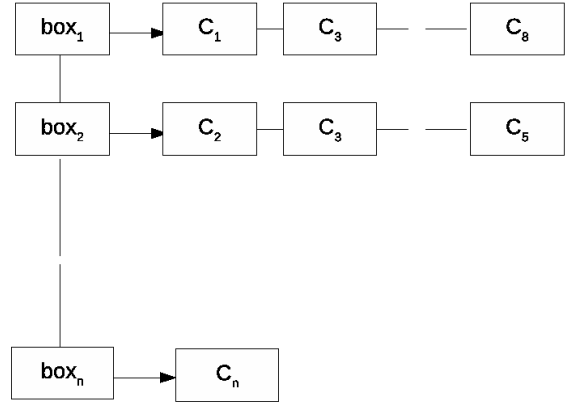


Fig. 12: Merging Apple Counts Across Frames. We keep track of the corresponding bounding boxes generated connected component analysis across multiple frames using homography. The count for every box from different frames is recorded throughout the entire video. In this example, box_1 appeared in frame 1, 3, ..., 8 and the counts for each of these frames $C_1, C_3 \dots C_8$ were recorded. The median counts for each box are reported as the final output of the counting method.

5 Experimental Results

In this section, we present experimental results validating our algorithms. We start with the datasets.

5.1 Datasets

To verify the performance of our algorithms, we collected multiple datasets. We group these datasets into two categories, namely - “Validation Datasets and Training Datasets”. As their names suggest, they are used for the purposes of validation, and training. All data were collected at the University of Minnesota Horticulture Research Center at Victoria Minnesota (Fig. 13), over the course of two years (2015 - 2016).

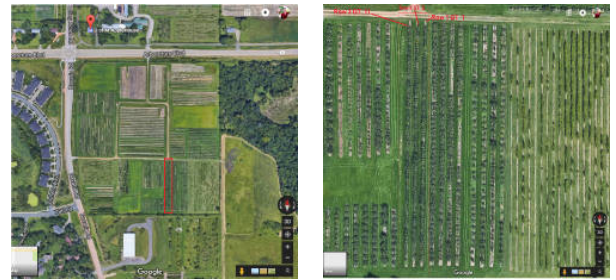


Fig. 13: Data collection site with annotated rows. The figure on left shows the orchard location and the figure on right shows the individual rows of apple trees.

Each of the videos collected from these datasets is tagged sunny/shady/cloudy based on the weather condition and whether the particular side of the row (captured in the video) was facing/opposing the sun.

Validation Datasets Since this is a research site, it is home to many different kinds of apple trees. We arbitrarily chose four different sections in the orchard. We collected seven videos from these four segments, all of which were annotated manually with fruit locations (annotation procedure will be discussed in the next section). We also collected ground truth for all of them by labeling the apples physically by stickers and measuring their diameter after harvest. The details of these datasets are the following:

Dataset1 This dataset contains six trees. Most of the apples on these trees were fully red and the trees were mostly planar (most of the apples are visible from both sides). We collected videos from both sides (the side facing the sun and opposing the sun) of the row. In total, there were 270 apples in these six trees. See Fig. 14 (leftmost) for a sample image from this dataset.

Dataset2 This dataset contains four trees. The trees had a mixture of red and green apples and complex (non-planar) geometry. We collected a video from a single side of the row (the side facing the sun). In total, there were 568 apples in these four trees. See Fig. 14 (second from left) for a sample image from this dataset.

Dataset3 This dataset contains ten trees. Apples in these trees were mostly red and the trees had non-planar geometry. In total, there were 274 apples in these ten trees. We collected videos from both sides (the side facing the sun and opposing the sun) of the row. See Fig. 14 (third from left) for a sample image from this dataset.

Dataset4 This dataset contains six trees. Fruits in these trees were a mixture of red and green apples and the trees had non-planar geometry. We collected videos from both sides of the row (the side facing the sun and opposing the sun). In total, there were 414 apples in these six trees. See Fig. 14 (fourth from left) for a sample image from this dataset.

All of the videos collected from these datasets are used to validate our segmentation and counting algorithms. Additionally, Dataset 1, 3 and 4 are used to verify the total fruit counts from both sides. As we only collected video from a single side of the row from Dataset2, it is not used to verify both side fruit counts.

Training Datasets To validate how our algorithms perform without user supervision, we picked a dataset

from 2015. This dataset contains 76 trees. Fruits in these trees were a mixture of red and green apples and the trees had non-planar geometry. We collected a video from a single side of a row (it was a cloudy day, both sides were illuminated similarly). This video is not annotated manually and ground truth for this dataset is unknown. We only used this dataset for developing a model to identify apples without any user intervention. In other words, the user input was provided for the 2015 dataset and used without modification for 2016 dataset.

The four validation datasets were captured using a Samsung Galaxy S4 camera in September 2016. The training dataset was collected in 2015 using a Garmin VR camera. It is notable that, our data collection facility is not a commercial orchard. Consequently, there is a great amount of variability in the shape of the trees even within the same crop row, which makes the yield estimation problem harder. In the next section, we look at the process of annotating the datasets.

5.2 Manual Annotation of Apples for Verifying Detection and Counting

In order to validate our segmentation and counting methods, we need image level ground truth (detected apples in each individual input image). This is different from the number of harvested apples from trees. For this purpose, we annotated the boundary of individual apples in the input images. This provides us with the ability to compare the bounding boxes generated by our algorithm with bounding boxes drawn manually.

For manual annotation, frames were selected arbitrarily every 1 – 3 second for the test videos (frame rate 30 fps), depending on how much the camera moved since the last annotated frame. For each of these frames, apples were tagged as *clearly visible* or *marginally visible* based on visibility guidelines (Fig. 15). An apple was considered clearly visible if more than half of its cross-sectional area and more than half of its perimeter were unoccluded. Otherwise, if it was still detected as an apple by the human, it was marked as marginally visible. The marginally visible apples have more ambiguous bounding boxes, and might not even have a one-to-one mapping between boxes and apples. In addition to these guidelines, apples that were growing on trees in the rows behind the main row of interest, and apples that had fallen to the ground, were not tagged.

Seven videos collected from the validation datasets (Dataset1 to Dataset4), described in the previous section, were tagged in this manner. The manually drawn bounding boxes were then propagated to the rest of the frames using camera motion between the frames. In the next sections, we investigate the performance of our



Fig. 14: Sample images from used datasets. First four figures show sample images from the test datasets and the last one shows a sample image from our training dataset.

segmentation and counting algorithms using these hand labeled datasets. Additionally, we will look at the relationship of computed yield vs actual yield for Dataset1, Dataset2, and Dataset4.

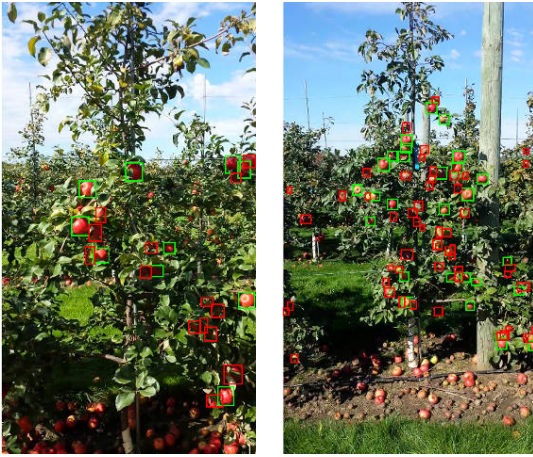


Fig. 15: Manual annotation of clearly visible and marginally visible apples for two sample images. An apple was considered *clearly visible* if more than half of its cross-sectional area and more than half of its perimeter were unoccluded (green boxes). Otherwise, if it was still detected as an apple by the human, it was marked as *marginally visible* (red boxes).

Our segmentation algorithm detects the ground apples as well. The manual annotation procedure ignores them. To remove ground apples, we let users choose a single line at the start of each video. This line is propagated using homography for the entire image sequence and any apple detected below this line are labeled as ground apples, and they are ignored for both segmentation and counting.

5.3 Performance Evaluation of the Segmentation Method

In this section, we study the performance of our segmentation method. In particular, we investigate its sensitivity with respect to user supervision across the validation datasets. We use three metrics, precision, recall and F_1 -measure for this purpose. As is well-known in literature, these metrics are obtained using true positives (TP), false positives (FP) and false negatives (FN). Formally, $\text{precision} = \frac{TP}{TP+FP}$ and $\text{recall} = \frac{TP}{TP+FN}$ and $F_1 \text{ measure} = 2 \left(\frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \right)$. We define TP, FP and FN using manually tagged apples and the apples detected by our algorithm. Specifically, TP = apples detected by our algorithm and tagged manually, FP = apples detected by the algorithm but not tagged manually, FN = apples tagged manually but not detected by our algorithm.

We compare the bounding boxes computed by our algorithm to manually drawn bounding boxes (guidelines for the manual annotation procedure was described in the previous section). To evaluate the quality of a particular bounding box generated by our algorithm, we use a metric well-known in the literature as the intersection over union (IoU) threshold (Hariharan et al. (2014)). This method factors in how much of each bounding box is detected by our algorithm (Fig. 16(a)). We demonstrate the performance of our algorithm in terms of precision, recall and F_1 -measure over the entire range of intersection over union threshold. For the purposes of counting though, we utilize a nominal nonzero intersection over union threshold (0.01).

One of the advantages of our algorithm is that, we do not detect all the apples on a single image. We utilize multiple views available from the video. It is evident

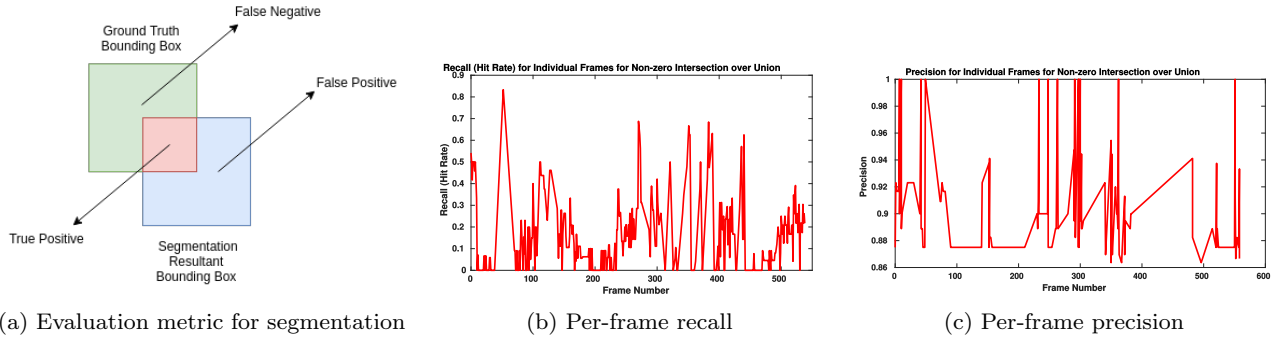


Fig. 16: Evaluation metric for segmentation, per-frame recall and precision. Figure on left shows how segmentation accuracy is evaluated by performing one to one comparison between the ground truth and our resultant bounding boxes. The figure on the middle, shows the variance in recall for per frame segmentation. The rightmost image shows that, our precision per-frame is always over 87%. Coupling this with multiple views we achieve high precision and recall (Fig. 17, 18, 19, 20) for the entire videos.

from Fig. 16(b) that for single frames even for nonzero intersection over union threshold (means if the algorithm detected bounding boxes just touches the ground truth), the per-frame apple detection rate (recall) varies significantly. Nevertheless, our per-frame precision is always over 87% and with the help of multiple views of each apple cluster we achieve high precision and recall (Fig. 17, 18, 19, 20).

In order to test the performance without user intervention, we trained a classification model from the dataset collected in 2015 and used this model for detecting apples from the validation dataset collected in 2016. To test the effect of user supervision on specific videos collected from datasets, we let users choose the apples from the first fifty frames of the input video. We utilize the clusters chosen by the users to build the classification model and use this model for segmenting apples. For the rest of the paper, we will refer to these two types of classification models as *semi-supervised* and *user-supervised* models.

First, we evaluate the computed recall for clearly and marginally visible apples. According to the visibility guidelines discussed in Section 5.2, the recall/hit rate for clearly visible apples should be higher than the recall for marginally visible apples. Fig. 17 shows, that this is indeed the case for both the *semi-supervised* and *user-supervised* classification models. We show the results for two different videos from the validation datasets (Dataset1 and Dataset2) in Fig. 17. For the figures on the top row, both models achieve high recall. For the figures on bottom row the *semi-supervised* model has a low recall and the *user-supervised* model has high recall. Importantly though, for all the figures we see that recall for clearly visible apples are higher than recall for marginally visible apples and the overall

recall is in between these two (especially for high IoU thresholds).

Second, we investigate the sensitivity of user input. For the *user-supervised* case, we only allow choosing apples for the first fifty frames. We see in Fig. 18,19 for the first five videos collected from Dataset1, Dataset3 and Dataset4 the precision and recall for both the *user-supervised* and *semi-supervised* models are similar. However, for the video collected from Dataset2, the apples are a mixture of red and green. Therefore, the *semi-supervised* classification model does not generalize well. Consequently, the precision is similar but the recall drops by 20%.

Finally, it is desired to obtain a single number associated with the performance of the algorithm. We use F_1 -measure for this purpose. This is the harmonic mean of precision and recall. Our F_1 -measure ranges from .95 – .97 for six videos (Fig. 20, Table 1) for nonzero intersection over union threshold. The best known F_1 -measure .91 was obtained by (Bargoti and Underwood (2016)). As the datasets are different, and our method does not maximize detection for a single image, a direct comparison is not possible. In the next section, we investigate the performance of our counting algorithm.

5.4 Performance Evaluation of the Counting Method

In this section, we quantify the performance of our counting method. We investigate how the algorithm (per-frame counting) performs on individual segmented images and how the overall performance (merging counts across frames) compares to human-perceived counts and ground truth. It is notable that following the segmentation step, all the bounding boxes with above nonzero

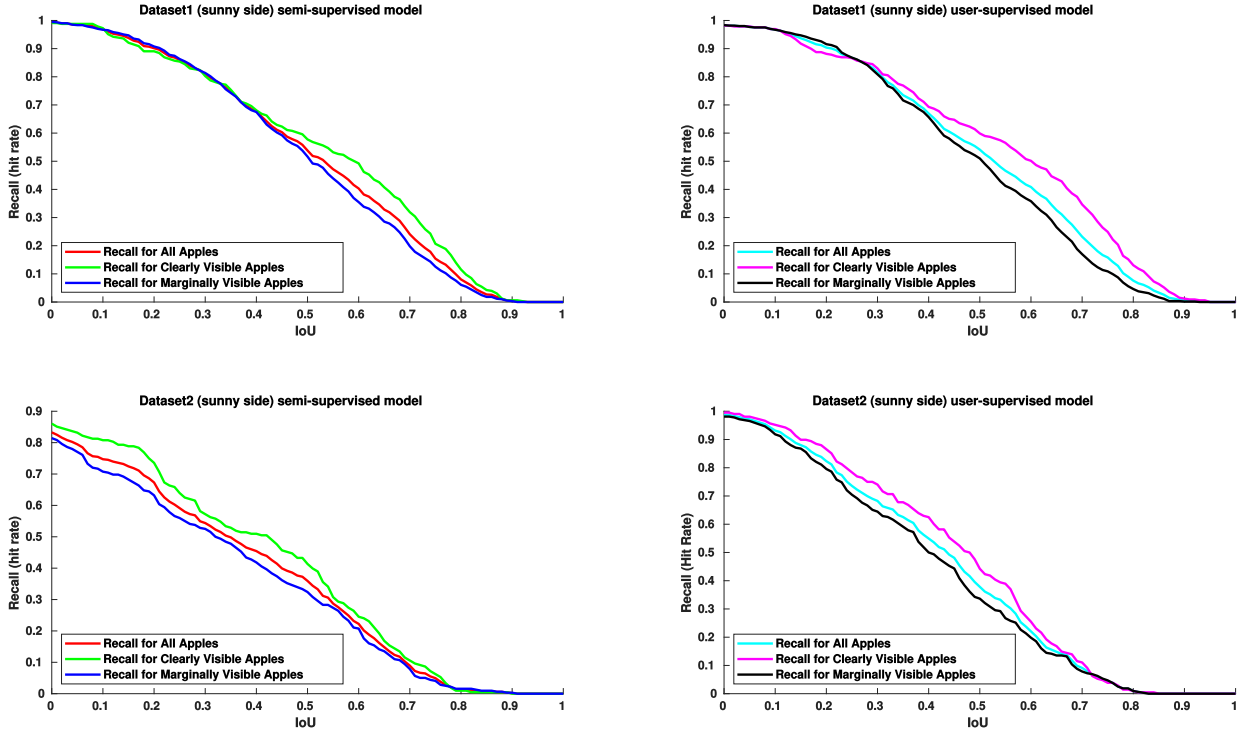


Fig. 17: Recall (hit rate) for clearly visible, marginally visible and all apples for both semi-supervised and *user-supervised* case. As expected, the recall for clearly visible apples are high, and the recall for marginally visible apples are low. The overall recall is in between this low and high bound.

Datasets	<i>Semi-supervised</i>	<i>User-supervised</i>
D_1 (Sunny)	.9662	.9658
D_1 (Shady)	.9585	.9609
D_3 (Sunny)	.9738	.9711
D_3 (Shady)	.9541	.9592
D_4 (Sunny)	.9774	.9775
D_2 (Sunny)	.8931	.9710

Table 1: F_1 - Measure for the Validation Dataset for IoU = .01. Here, D_1, D_2, D_3, D_4 denote Dataset1, Dataset2, Dataset3 and Dataset4.

intersection over union threshold are used in the counting step.

For evaluating both per frame counting and merging, we utilize the videos collected from the validation datasets (Dataset1 - Dataset4). We start with the per-frame counting method.

5.4.1 Evaluation of Per-frame Counting Method:

To evaluate the performance of the per-frame counting algorithm, we took all the segmented images from seven videos collected from the four datasets. Afterward, we

performed a connected component analysis on them, randomly selected 5000 components and marked each one with the perceived count from a human point of view. These counts are then compared to the counts obtained from the algorithm. At this stage, we want the segmented images to be accurate and consequently, we use the *user-supervised* model for segmentation. Essentially, we have three key insights from this experiment. First, it is evident from the confusion matrix (Fig. 21(a)), that recall drops with increasing cluster size (varies from 62.7% – 99%) but precision stays over 87% for any cluster size (varies from 87.8% – 96%). Second, for a large portion of the data - single apples (75.31% of entire data (Fig. 21(b))); the precision and recall of our algorithm are 96% and 99% respectively. Consequently, the overall accuracy of our method is high (94.4%) (shown in the right-bottom cell in the confusion matrix). Third, low recall rates for larger clusters does not affect the overall performance. In the next section, utilizing the high precision and multiple views we achieve good recall on the entire data (Fig. 22(b)).

Next, we quantify the effect of lighting conditions (sunny, shady, cloudy etc.) on the counting method. We computed the accuracy of the per-frame counting

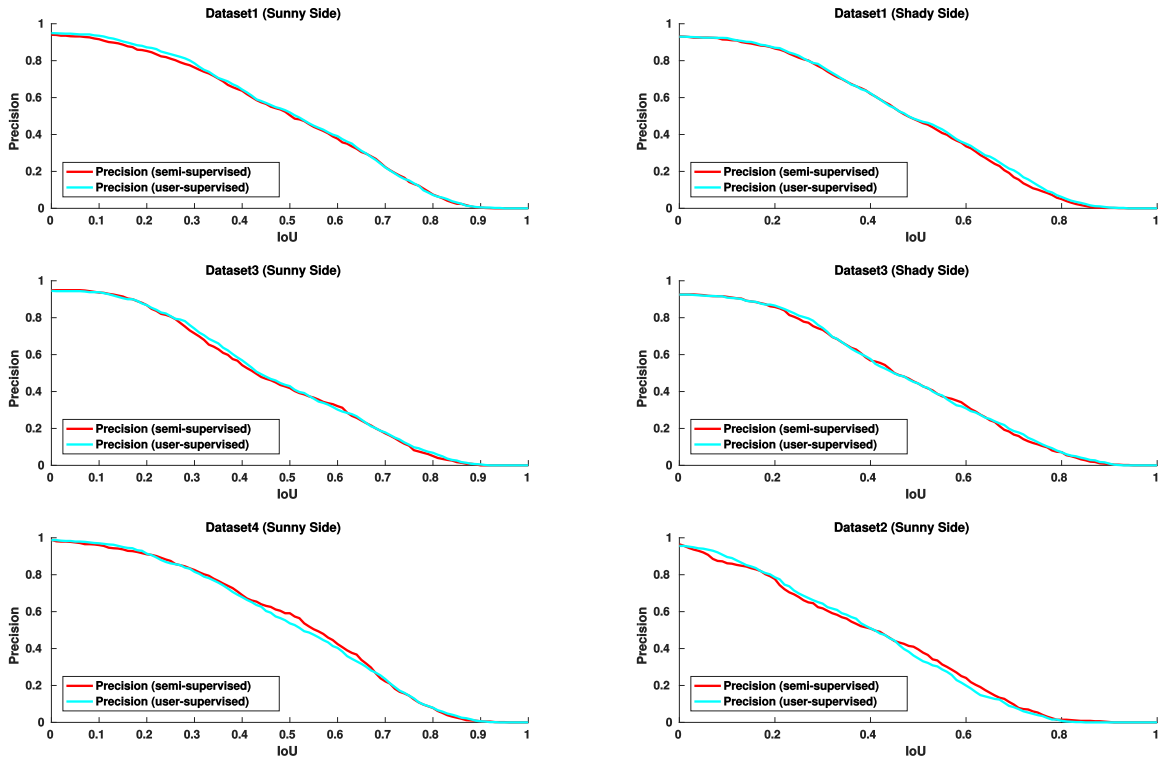


Fig. 18: Precision for *semi-supervised* and *user-supervised* models. For all the videos, the obtained precision is similar for both models.

method across all the collected videos (which were collected in different lighting conditions). Our counting accuracy varied from 94.01% – 95.38%. Undercounting percentage varies from 3.62% – 5.67% and overcounting varies from .74% – 1.1%. These results are presented in Fig. 21(c). In the next section, we evaluate the performance of merging the apple counts using camera motion.

5.4.2 Merging Counts Across Multiple Frames

To verify the performance of the merging method, we utilize the manual annotations. We treat the manually annotated fruits as human perceived ground truth. Afterward, we track these fruits across frames using camera motion (3D camera poses) to avoid double counting and find the number of unique apples. The counts obtained in this manner are then compared to the counts from our merging algorithm. First, we quantify the amount of overcounting due to approximating the camera motion using homography. We performed this evaluation by simply comparing the number of unique manually labeled apples obtained from homography to the number of unique apples obtained by utilizing full 3D camera motion. The camera motions were computed using a

commercial photogrammetry software Agisoft (Agisoft (2017)). We found that across different datasets; using homography can lead to 8% overcounting. Fig. 22(a) shows these results.

Next, we evaluate the accuracy of the merged counts (tracking by homography) to human-perceived counts (tracking by 3D camera poses). To understand the importance of user interaction, we perform this analysis for both *semi-supervised* and *user-supervised* models (Fig. 22(b)). Our accuracy with respect to human perceived ground truth varies from 89% – 98% for the *user-supervised* model and 48% – 98% for the *semi-supervised* model. The drop in accuracy for the semi-supervised model was propagated from the segmentation phase. The main takeaways from these results are, 1) accurate segmentation is very important for obtaining correct counts. 2) based on the geometry of the environment and lighting conditions, we count 89%–98% of the visible apples from a single side. In the next section, we investigate how the single side counts correlate with the actual yield.

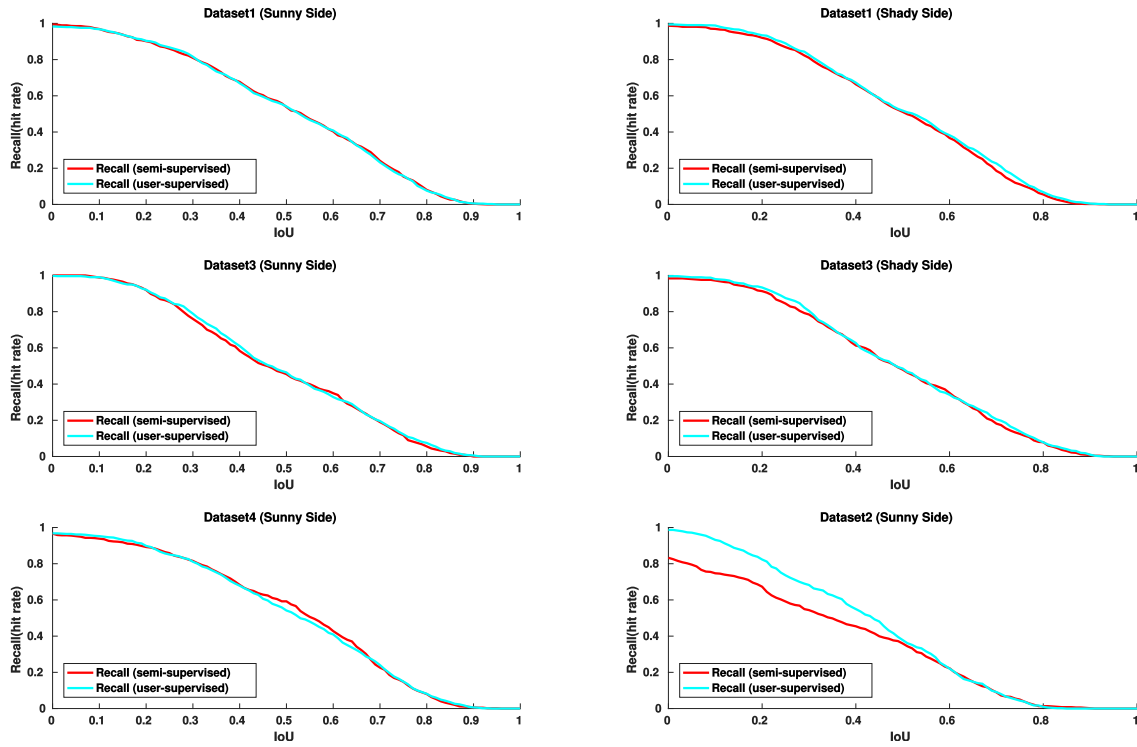


Fig. 19: Recall for *semi-supervised* and *user-supervised* models. For the five videos (collected from Dataset1, Dataset3, Dataset4) the recall for both models is similar. For the last video (collected from Dataset2) the recall for the *semi-supervised* model drops by 20%. This is due to the fact that, the apples contained this video are a mixture of green and red which is not captured by the *semi-supervised* model representing primarily red apples.

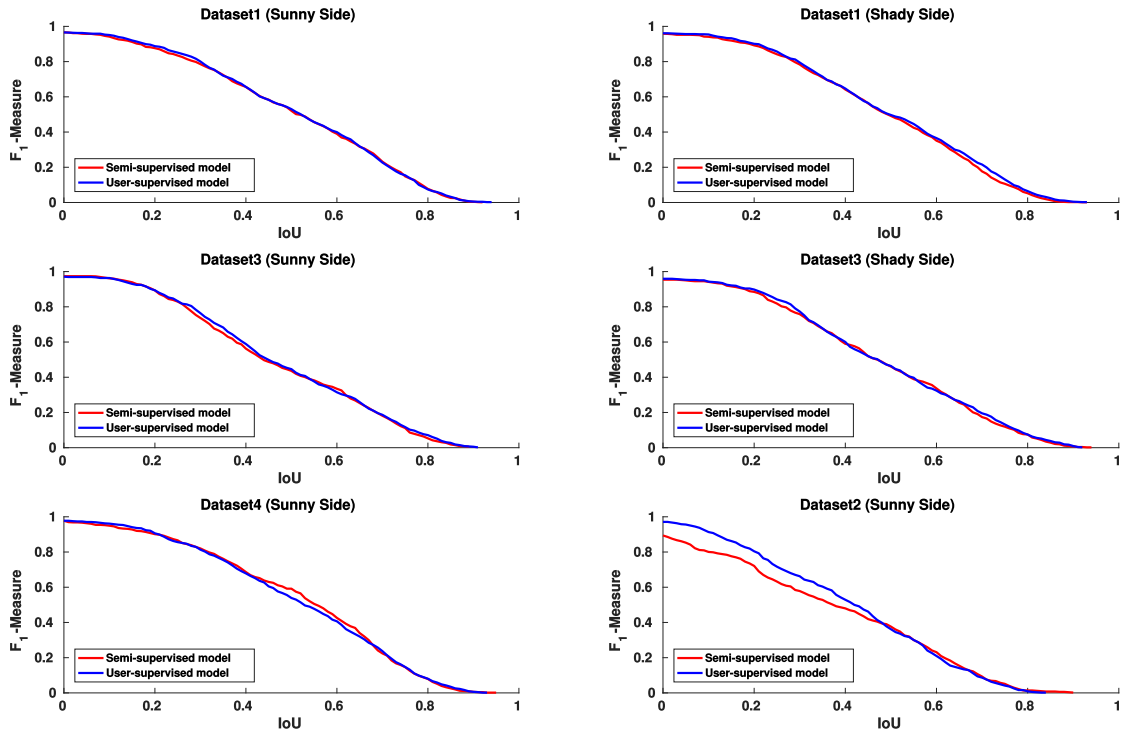
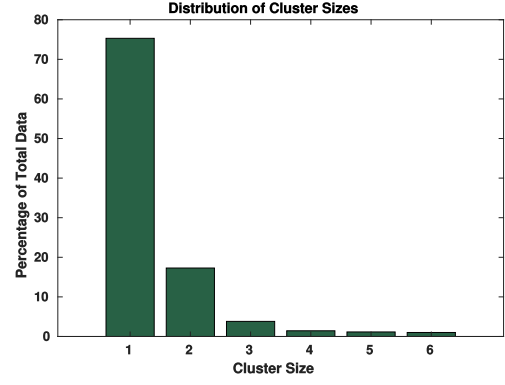


Fig. 20: F_1 - measure for both *semi-supervised* and *user-supervised* models.

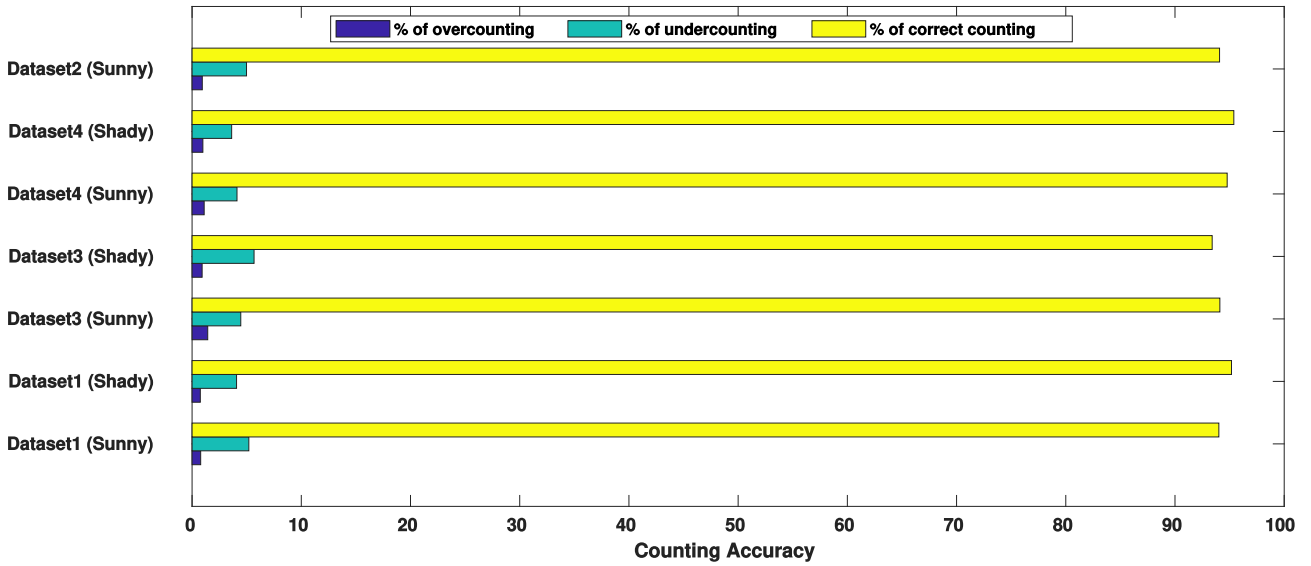
Confusion Matrix for Predicted and Ground Truth Apple Counts

Predicted Cluster Size \ Original Cluster Size	1	2	3	4	5	6	
1	3323 74.5%	119 2.7%	16 0.4%	1 0.0%	1 0.0%	0 0.0%	96.0% 4.0%
2	34 0.8%	651 14.6%	36 0.8%	9 0.2%	3 0.1%	0 0.0%	88.8% 11.2%
3	1 0.0%	1 0.0%	118 2.6%	7 0.2%	5 0.1%	0 0.0%	89.4% 10.6%
4	0 0.0%	0 0.0%	0 0.0%	43 1.0%	6 0.1%	0 0.0%	87.8% 12.2%
5	0 0.0%	0 0.0%	0 0.0%	2 0.0%	32 0.7%	2 0.0%	88.9% 11.1%
6	0 0.0%	0 0.0%	0 0.0%	2 0.0%	4 0.1%	43 1.0%	87.8% 12.2%
	99.0% 1.0%	84.4% 15.6%	69.4% 30.6%	67.2% 32.8%	62.7% 37.3%	95.6% 4.4%	94.4% 5.6%



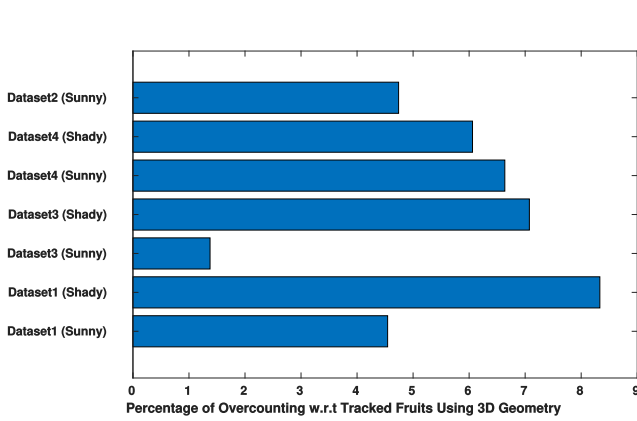
(a) Confusion matrix for the per-frame counting method.

(b) Distribution of cluster sizes.



(c) Accuracy of the per-frame counting method across different videos collected from Dataset1-Dataset4.

Fig. 21: Evaluating the performance of the per-frame counting algorithm- confusion matrix (left), distribution of cluster sizes (middle) and performance across different datasets (right). The right-bottom cell in the confusion matrix (figure on left) shows that the overall accuracy of our method is 94.4%. The rightmost column (Rows 1 – 6) shows the precision per cluster size and the bottom row (Columns 1 – 6) shows recall per cluster size. From the figure on the middle, we find that single apple clusters are dominating the data (75.31%). The figure on right shows how our counting accuracy varied from 94.01% – 95.38% across different videos.



(a) Evaluation of tracking by homography.

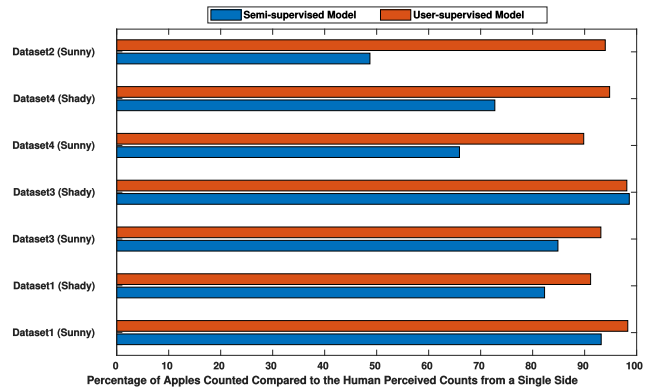
(b) Performance comparison of *semi-supervised* and *user-supervised* models for counting.

Fig. 22: Evaluating the performance of the counting algorithm along with tracking. The figure on left shows the percentage of overcounting due to using homography for tracking (up to 8%). Figure on the right shows, the percentage of visible apples counted by our method using both *semi-supervised* and *user-supervised* models.

5.5 Yield Estimation

Our original goal is to get an accurate yield estimate. For that, we need to correlate the counts from our algorithm (from the single side of a row) to the actual ground truth (number of harvested fruits). Toward this, we first try to find out a correlation between the number of visible apples from a single side and the actual yield. As mentioned earlier, we determine the total number of visible apples by tracking the manually labeled apples from a single side, utilizing estimated camera motion. For this step, we will use datasets where videos from both sides were collected (Dataset1, Dataset3, Dataset4). From Fig. 24(a), it is evident that the number of visible apples from a single side vary greatly across different datasets (40.85% – 79.83%). This is expected as the orchard from which we collected the data is not well trimmed and the size and shape of the trees varies significantly even within the same row.



Fig. 23: Merged reconstruction for Dataset1.

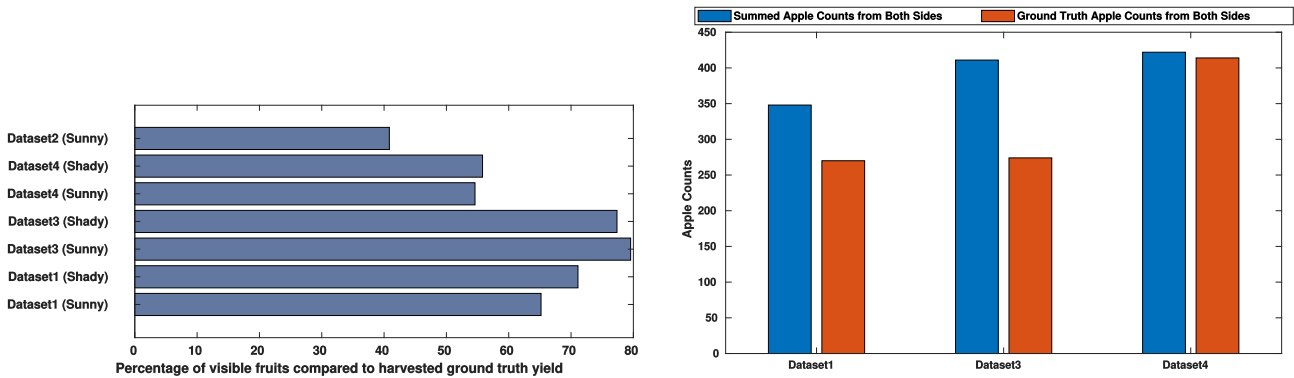
Another simple solution is adding the apple counts from both sides and finding a correlation with the ac-

tual yield. Fig. 24(b) shows these results and again, the summed yields vary considerably across datasets (101.93% – 150%). Therefore, to get a useful estimate we need to merge the fruit counts from both sides of the tree rows. Next, we discuss this procedure in details.

To merge counts from both sides, we utilize the 3D geometry of the environment. We reconstruct each side of the row using captured images. We use a photogrammetry software Agisoft (Agisoft (2017)) for this purpose. Afterward, we merge the reconstructions from both sides using semantic constraints (Dong and Isler (2018); Roy et al. (2018)). Fig. 23 shows an example (merged reconstruction for Dataset1).

Next, we detect the fruits using our segmentation method (Section 3) and back-project the detected fruits in the images to obtain the fruit location in the 3D reconstruction. Fig. 25 shows an example. We perform a connected component analysis to detect the apple clusters in 3D. Then we project individual 3D clusters back to the images by utilizing the recovered camera motion. We count the fruits from these reprojected images using our counting method developed in Section 4. A 3D cluster can be tracked over many frames. We choose three frames with the highest amount of detected apple pixels (from the 3D cluster) and report the median count of these three frames as the fruit count for the cluster. We follow this procedure for all the detected 3D clusters and aggregate the fruit count from a single side.

To merge the counts from both sides, we compute the intersection of the connected components from both sides. Afterward, we compute the total counts by using the inclusion-exclusion principle (Andreescu and Feng (2004)). Essentially, we sum up the counts from



(a) Percentage of visible apples from a single side compared to ground truth yield.

(b) Summed counts from single sides compared to obtained ground truth.

Fig. 24: Correlation between single side fruit counts and actual yield. The figure on left shows the percentage of visible apples from a single side compared to ground truth (40.85% – 79.83%). Figure on the right shows the summed fruit counts from both sides compared to ground truth fruit counts (101.93% – 150%).

all the connected components, compute the intersections area among them and add/subtract the weighted parts accordingly. Fig. 26 shows our result. Our counting accuracy from both sides for Dataset1, Dataset3 and Dataset4 are 94.81%, 91.98%, and 94.68% respectively. Compared to both side counts, if we just add the single side counts we overcount significantly (128.89%, 150%, and 101.93% for Dataset1, Dataset3, and Dataset4 respectively). Table 2 summarizes the final yield result. It indicates that merging the rows from both sides is essential to obtain accurate yield.

6 Conclusion and Future Work

In this paper, we presented a complete yield estimation system for apple orchards from monocular images. From a purely technical point of view, our main contributions are a semi-supervised clustering method relying on color for identifying the apples and an unsupervised clustering method relying on shape to estimate the number of apples in a cluster. We verified the performance of our algorithms on multiple small datasets. Results indicate that these algorithms perform well in practice and outperform most of the existing methods in terms of detection and counting accuracy.

As reported in section 5.4, we count 89% – 98% of the visible apples from a single side of the row in different datasets. To be of practical usage though, we needed to correlate this single side counts with harvested yield. With the help of our recent work (Dong and Isler (2018); Roy et al. (2018)), we merged the fruit counts from both sides of fruit tree rows. Our method achieved a varying accuracy of 91.98% – 94.81% across different datasets.

In future, we would like to couple our system more closely with the 3D geometry of the environment. We would like to develop techniques to localize each individual apple in a cluster, find the pose of the fruit and measure its diameter.

Acknowledgement

The authors thank Joshua Anderson, Professors Emily Hoover, and Cindy Tong from the Department of Horticultural Science, University of Minnesota, for their expertise and help with the experiments. This work is supported in part by NSF grant # 1317788, USDA NIFA MIN-98-G02 and the MnDrive initiative.

References

References

- Achanta R, Shaji A, Smith K, Lucchi A, Fua P, Süsstrunk S (2012) Slic superpixels compared to state-of-the-art superpixel methods. *IEEE transactions on pattern analysis and machine intelligence* 34(11):2274–2282
- Agisoft L (2017) Agisoft PhotoScan. <http://www.agisoft.com/>, accessed: 2017-09-15
- Andreescu T, Feng Z (2004) Inclusion-exclusion principle. In: *A Path to Combinatorics for Undergraduates*, Springer, pp 117–141
- Baker S, Matthews I (2004) Lucas-kanade 20 years on: A unifying framework. *International journal of computer vision* 56(3):221–255

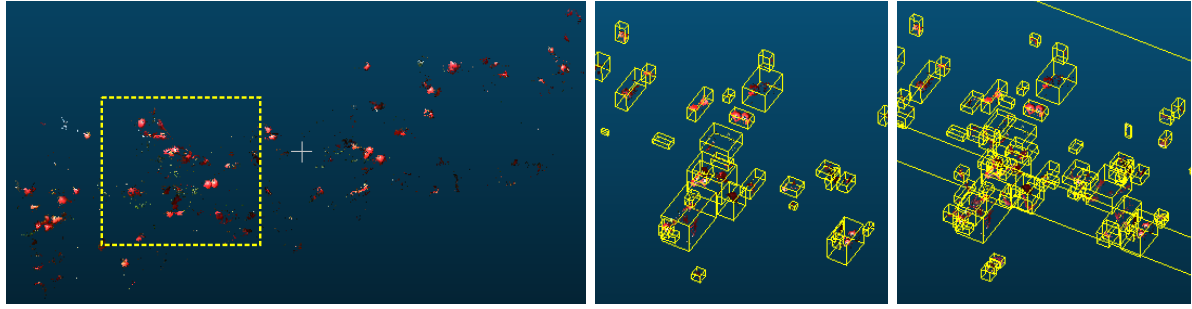


Fig. 25: Apples in 3D and intersecting clusters from both sides. Left figure shows the apples from the front side of Dataset1 detected in 3D. Figure in the middle shows the computed connected components on this apples. Figure in the right shows how the connected components from the back are intersecting with the front ones.

	Harvested fruit counts	Merged fruit counts from both sides	Sum of fruit counts from single sides
Dataset1	270	256 (94.81%)	348 (128.89%)
Dataset3	274	252 (91.98%)	411 (150%)
Dataset4	414	392 (94.68%)	422 (101.93%)

Table 2: Summary of yield results.

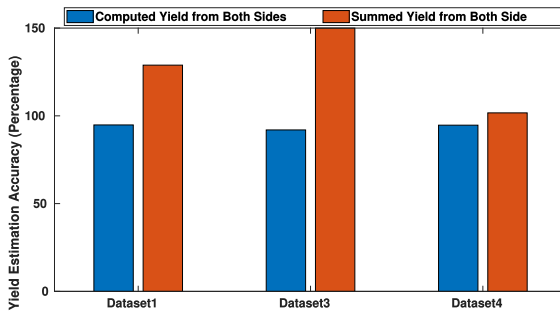


Fig. 26: Total fruit counts computed from the merged reconstruction and independent single side reconstructions compared to ground truth. The single side fruit counts are computed by summing up independent single side counts (101.93% – 150%). Evidently, the counts from merged reconstructions are more consistent (91.98% – 94.81%).

Bargoti S, Underwood JP (2016) Image segmentation for fruit detection and yield estimation in apple orchards. CoRR abs/1610.08120

Bargoti S, Underwood JP (2017) Image segmentation for fruit detection and yield estimation in apple orchards. Journal of Field Robotics

Bilmes JA, et al. (1998) A gentle tutorial of the em algorithm and its application to parameter estimation for gaussian mixture and hidden markov models. International Computer Science Institute 4(510):126

Changyi X, Lihua Z, Minzan L, Yuan C, Chunyan M (2015) Apple detection from apple tree image based on bp neural network and hough transform. Interna-

tional Journal of Agricultural and Biological Engineering 8(6):46–53

Chen SS, Gopalakrishnan PS (1998) Clustering via the bayesian information criterion with applications in speech recognition. In: Acoustics, Speech and Signal Processing, 1998. Proceedings of the 1998 IEEE International Conference on, IEEE, vol 2, pp 645–648

Chen SW, Shivakumar SS, Dcunha S, Das J, Okon E, Qu C, Taylor CJ, Kumar V (2017) Counting apples and oranges with deep learning: A data-driven approach. IEEE Robotics and Automation Letters 2(2):781–788

Chuang YY, Curless B, Salesin DH, Szeliski R (2001) A bayesian approach to digital matting. In: Computer Vision and Pattern Recognition, 2001. CVPR 2001. Proceedings of the 2001 IEEE Computer Society Conference on, IEEE, vol 2, pp II–II

Das J, Cross G, Qu C, Makineni A, Tokekar P, Mulgaonkar Y, Kumar V (2015) Devices, systems, and methods for automated monitoring enabling precision agriculture. In: Proceedings of IEEE Conference on Automation Science and Engineering

Dong W, Isler V (2018) Tree morphology for phenotyping from semantics-based mapping in orchard environments. Tech. rep., Technical Report TR-18-006, University of Minnesota, Computer Science & Engineering Department

Eshel R, Moses Y (2008) Homography based multiple camera detection and tracking of people in a dense crowd. In: Computer Vision and Pattern Recognition, 2008. CVPR 2008. IEEE Conference on, IEEE,

- pp 1–8
- Gauch JM, Hsia CW (1992) Comparison of three-color image segmentation algorithms in four color spaces. In: Applications in optical science and engineering, International Society for Optics and Photonics, pp 1168–1181
- Goldberger J, Gordon S, Greenspan H, et al. (2003) An efficient image similarity measure based on approximations of kl-divergence between two gaussian mixtures. In: ICCV, vol 3, pp 487–493
- Gongal A, Silwal A, Amatya S, Karkee M, Zhang Q, Lewis K (2016) Apple crop-load estimation with over-the-row machine vision system. *Computers and Electronics in Agriculture* 120:26–35
- Grunwald P (2004) A tutorial introduction to the minimum description length principle. arXiv preprint math/0406077
- Hariharan B, Arbeláez P, Girshick R, Malik J (2014) Simultaneous detection and segmentation. In: European Conference on Computer Vision, Springer, pp 297–312
- Hartigan JA, Wong MA (1979) Algorithm as 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society Series C (Applied Statistics)* 28(1):100–108
- Hecht-Nielsen R (1989) Theory of the backpropagation neural network. In: Neural Networks, 1989. IJCNN., International Joint Conference on, IEEE, pp 593–605
- Hung C, Underwood J, Nieto J, Sukkarieh S (2015) A feature learning based approach for automated fruit yield estimation. In: Field and Service Robotics, Springer, pp 485–498
- Jimenez A, Ceres R, Pons J (2000) A survey of computer vision methods for locating fruit on trees. *Transactions of the ASAE-American Society of Agricultural Engineers* 43(6):1911–1920
- Linker R, Cohen O, Naor A (2012) Determination of the number of green apples in rgb images recorded in orchards. *Comput Electron Agric* 81:45–57
- Liu X, Zhao D, Jia W, Ruan C, Tang S, Shen T (2016) A method of segmenting apples at night based on color and position information. *Computers and Electronics in Agriculture* 122:118–123
- Long J, Shelhamer E, Darrell T (2015) Fully convolutional networks for semantic segmentation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp 3431–3440
- Lowe DG (1999) Object recognition from local scale-invariant features. In: Computer vision, 1999. The proceedings of the seventh IEEE international conference on, Ieee, vol 2, pp 1150–1157
- Pedersen SJK (2007) Circular hough transform. Aalborg University, Vision, Graphics, and Interactive Systems 123
- Roy P, Isler V (2016) Surveying apple orchards with a monocular vision system. In: Automation Science and Engineering (CASE), 2016 IEEE International Conference on, IEEE, pp 916–921
- Roy P, Isler V (2017) Vision-based apple counting and yield estimation. In: Kulić D, Nakamura Y, Khatib O, Venture G (eds) 2016 International Symposium on Experimental Robotics, Springer International Publishing, Cham, pp 478–487
- Roy P, Dong W, Isler V (2018) Registering reconstructions of the two sides of fruit tree rows. Tech. rep., Technical Report TR-18-008, University of Minnesota, Computer Science & Engineering Department
- Ruzon MA, Tomasi C (2000) Alpha estimation in natural images. In: Computer Vision and Pattern Recognition, 2000. Proceedings. IEEE Conference on, IEEE, vol 1, pp 18–25
- Senthilnath J, Dokania A, Kandukuri M, Ramesh K, Anand G, Omkar S (2016) Detection of tomatoes using spectral-spatial methods in remotely sensed rgb images captured by uav. *Biosystems Engineering*
- Silwal A, Gongal A, Karkee M (2014) Apple identification in field environment with over the row machine vision system. *Agricultural Engineering International: CIGR Journal* 16(4):66–75
- Sinha SN, Steedly DE, Szeliski RS (2014) Multi-stage linear structure from motion. US Patent 8,837,811
- Stein M, Bargoti S, Underwood J (2016) Image based mango fruit detection, localisation and yield estimation using multiple view geometry. *Sensors* 16(11):1915
- Tabb AL, Peterson DL, Park J (2006) Segmentation of apple fruit from video via background modeling. In: 2006 ASAE Annual Meeting, American Society of Agricultural and Biological Engineers, p 1
- Wang Q, Nuske S, Bergerman M, Singh S (2013) Automated crop yield estimation for apple orchards. In: Desai JP, Dudek G, Khatib O, Kumar V (eds) Experimental Robotics, Springer Tracts in Advanced Robotics, vol 88, Springer International Publishing, pp 745–758
- Zhou R, Damerow L, Sun Y, Blanke MM (2012) Using colour features of cv. ‘gala’ apple fruits in an orchard in image processing to predict yield. *Precision Agriculture* 13(5):568–580